

ESTRUCTURAS DE DATOS

Práctica de Laboratorio N° 10: Eliminación en Árboles Binarios de Búsqueda

I. OBJETIVOS DE LA PRÁCTICA:

- Comprender el Árbol Binario de Búsqueda (ABB) como una estructura de datos dinámica, no lineal y recursiva, que permite organizar información jerárquicamente para facilitar operaciones eficientes de búsqueda, inserción y eliminación.
- Comparar las ventajas del ABB frente a otras estructuras de datos, reconociendo su eficiencia y aplicabilidad en la resolución de problemas que requieren acceso rápido y ordenado a los datos.
- Implementar y analizar los recorridos inorden, preorden y posorden, así como algoritmos de inserción y búsqueda en un ABB, utilizando tanto métodos recursivos como iterativos.

II. DESARROLLO DE LA PRÁCTICA:

En esta práctica se implementará el algoritmo de eliminación en un árbol binario de búsqueda (ABB), tomando en cuenta lo siguiente:

Eliminación en un Árbol Binario de Búsqueda (Cairó Guardati, 2006):

La operación de **eliminación en un árbol binario de búsqueda** es un poco más complicada que la de inserción. Esta consiste en eliminar un nodo sin violar los principios que definen un árbol binario de búsqueda. Se deben distinguir los siguientes casos:

1. **Si el elemento a eliminar es terminal u hoja**, simplemente se suprime redefiniendo el puntero de su predecesor.
2. **Si el elemento a eliminar tiene un solo descendiente**, entonces tiene que sustituirse por ese descendiente.
3. **Si el elemento a eliminar tiene los dos descendientes**, entonces se tiene que sustituir por el nodo que se encuentra más a la izquierda en el subárbol derecho o por el nodo que se encuentra más a la derecha en el subárbol izquierdo.

Cabe destacar que antes de eliminar un nodo, se debe localizar éste en el árbol. Para esto se utiliza el algoritmo de búsqueda presentado anteriormente.

```
132 void Arbol::eliminarABB(Nodo*& apnodo, int dato)
133 {
134     if(apnodo != NULL){
135         if(dato < apnodo->info){
136             eliminarABB(apnodo->izq, dato); // Buscar en subárbol izquierdo
137         }else if(dato > apnodo->info){
138             eliminarABB(apnodo->der, dato); // Buscar en subárbol derecho
139         }else{
140             Nodo* otro = apnodo;
141             // Caso 1: Es una hoja
142             if(apnodo->izq == NULL && apnodo->der == NULL){
143                 apnodo = NULL;
144             }
145             // Caso 2: Tiene solo hijo izquierdo
146             }else if(apnodo->der == NULL){
147                 apnodo = otro->izq;
148             }
149             // Caso 2: Tiene solo hijo derecho
150             }else if(apnodo->izq == NULL){
151                 apnodo = otro->der;
152             }
153             }else{
154                 // Caso 3: Tiene dos hijos
155                 Nodo* aux = apnodo->izq;
156                 Nodo* aux1 = NULL;
157                 bool bo = false;
158                 // Buscar el mayor de los menores (extremo derecho del subárbol izquierdo)
159                 while(aux->der != NULL){
160                     aux1 = aux;
161                     aux = aux->der;
162                     bo = true;
163                 }
164                 apnodo->info = aux->info; // Reemplazar info del nodo a eliminar
165                 otro = aux;
```

ESTRUCTURAS DE DATOS

```
163 // Reconectar el subárbol izquierdo
164 if(bo == true){
165     aux1->der = aux->izq;
166 }else{
167     apnodo->izq = aux->izq;
168 }
169 }
170 delete otro; // Eliminar nodo
171 }
172
173 }else{
174     cout << "La información a eliminar no se encuentra en el árbol"<< endl;
175     return;
176 }
177 }
```

Ejercicio:

Implemente un método llamado `podarArbol` que elimine todos los nodos del árbol binario de búsqueda, liberando correctamente la memoria utilizada por cada nodo. Este procedimiento debe recorrer el árbol y eliminar recursivamente todos sus elementos, dejando la raíz apuntando a NULL.

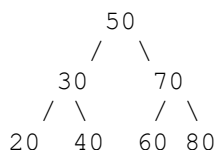
Este proceso se conoce como poda total del árbol, y es útil, por ejemplo, cuando se desea liberar completamente la estructura antes de finalizar un programa o reiniciar el contenido del árbol.

Ejemplo de entrada:

Se tiene el siguiente árbol con los valores insertados en este orden:

50, 30, 70, 20, 40, 60, 80

La estructura del árbol sería:



Salida esperada tras ejecutar `podarArbol`:

- Todos los nodos deben ser eliminados de memoria.
- El apuntador a la raíz debe quedar en NULL.
- Si luego se imprime el árbol, debe mostrarse como vacío.